

RAMANGASON Notahiana Erwan
CORDERO Adrien

Rapport Projet Système Informatique

Développement du compilateur	3
Utiliser le programme	3
Analyseur lexical et syntaxique	3
Analyseur lexical	3
Les tokens	3
Les règles	3
Variables	4
La structure	4
Le stockage des variables	4
Variables temporaires	4
Création et modification des variables	5
Opérations	5
Boucles et conditions	5
La condition (if) et la boucle while	5
If-else	6
Ecriture dans le fichier assembleur	6
Conception du microprocesseur	7
Tester le microprocesseur	7
Unité Arithmétique et Logique (UAL)	7
L'addition	7
La soustraction et la division	7
La multiplication	7
Banc de registres	8
Mémoire pour les données	8
Mémoire pour les instructions	8
Chemin de données	9
Faire le lien entre les composants	9
Ajout de l'instruction NOP	9

Développement du compilateur

Dans cette première partie du projet nous allons utiliser LEX et YACC pour développer un compilateur qui comprend quelques instructions du langage C.

Utiliser le programme

Pour utiliser le programme, vous trouverez un fichier Makefile dans le répertoire. La compilation du code se fait avec la commande “make”. Une fois compilé, il faut mettre le code C dans un fichier “test.c” qui se trouve dans le même répertoire et lancer la commande “make run”.

Analyseur lexical et syntaxique

Analyseur lexical

Notre programme ne reconnaît qu’une petite partie des symboles connus par C. Le fichier ‘c.l’ (le fichier lex) envoie au YACC (fichier ‘c.y’) chaque caractère lu parmi ceux qu’il reconnaît.

Les tokens

YACC est ensuite capable de reconnaître ces éléments car on a créé pour chaque élément un token avec un nom commençant par ‘t’ puis le nom de l’élément (ex : tMAIN, tFI pour Fin Instruction (le point virgule), tEG...). Certains de ces tokens sont plus prioritaires que d’autres pour les opérations arithmétiques. On a fait cela avec l’instruction “%left”. Nous avons mis en premier l’addition et la soustraction, puis ensuite la multiplication et la division car les plus prioritaires doivent être placés après les moins prioritaires. Certains token ont également des types. C’est le cas de tNAME (chaîne de caractère) et tNB (integer). tIF et tWHILE sont également des integers.

Les règles

L’analyseur lexical est ensuite découpé en plusieurs règles. Ces règles permettent de faire une action à chaque fois qu’une syntaxe est détectée. Chaque règle appelle une fonction qui se trouve dans 2 autres fichiers C : “variables.c” et “asm.c” (les headers sont respectivement “variables.h” et “asm.h”). Par exemple, la règle “Main” permet de reconnaître la fonction main et s’écrit comme ceci :

Main :
 { asm_create_file() } tMAIN tPO tPF Bloc { generate_asm(); asm_close_file() }

Ici, lorsque le programme va lire la fonction main, il va dans un premier temps, créer le fichier assembleur pour pouvoir mettre toutes les instructions dedans, puis exécuter la règle Bloc (qui s’occupe des blocs entre accolades et des instructions à l’intérieur), puis enfin générer l’ensemble du fichier assembleur et le fermer.

Pour gérer les instructions, Il existe une règle "Instructions", qui permet d'avoir 0, ou plusieurs instructions. La règle Instruction quant-à-elle permet d'appeler la bonne règle qui exécute la bonne fonction par rapport à celle demandée. L'instruction peut être une variable, une condition, une boucle, un print ou un autre bloc.

Variables

La structure

La gestion des variables dans notre programme se fait principalement avec les fichier "variables.c" et "variables.h". Les seules types de variable que l'on retrouve sont les entiers et les pointeurs d'entier. On retrouve à l'intérieur du header une structure "var_int_t" correspondant à une variable. Elle contient le nom de la variable (maximum 64 caractères), son adresse et deux booléen pour savoir si la variable est constante ou si c'est un pointeur.

```
typedef struct {
    char name[64];
    int addr;
    bool is_const;
    bool is_pointer;
} var_int_t;
```

Le stockage des variables

Dans le fichier "variables.c", on retrouve un tableau de "var_int_t". C'est ici que sont stockées toutes les variables. Les tableaux en C ayant une taille fixe, nous avons choisi de doubler la taille de celui-ci à chaque fois qu'il est plein. Nous nous aidons pour cela de la variable global "max_var", qui correspond à la taille actuelle du tableau. Une autre variable global ("nb_var") permet de connaître le nombre de variable actuellement et donc de savoir à quel indice du tableau on peut placer la prochaine variable. Un simple test d'égalité et un realloc nous permettent ensuite de redimensionner le tableau.

```
if (nb_var == max_var) {
    max_var *= 2;
    table_var = realloc(table_var, max_var * sizeof(int));
}
```

Variables temporaires

Pour gérer les opérations, nous allons avoir besoin de variables temporaires. Ces variables sont stockées dans le même tableau que les autres variables. Cependant, elles n'ont pas de nom (Le paramètre "name" vaut ""). Leur valeur est stockée dans le paramètre "addr" qui est un entier.

Création et modification des variables

Pour utiliser la structure décrite ci-dessus, nous avons créé plusieurs fonctions pour déclarer (“decl” ou “decl_assign_const”), assigner (“assign”) une valeur ou lire le contenu d’une variable (“get_value” ou “get_value_pointer”).

Opérations

Les opérations sont reconnues par le fichier yacc à l’aide de la règle “Terme”. Cette règle a le type int. Cette valeur représente l’adresse à laquelle le résultat est stocké. Cela lui permet d’être simplement un nombre, mais aussi le résultat d’une opération. Les opérations peuvent donc être les unes dans les autres. La priorité donnée aux opérateurs (voir partie Analyseur local et syntaxique) nous permet de tout gérer avec une seule règle.

Lorsqu’on croise un nombre, on va lui créer une variable temporaire pour qu’il soit traité au même niveau que les variables. Cela facilite l’intégration des variables dans les opérations.

Terme:

```
tNB { $$ = create_tmp($1); }  
| Terme tADD Terme { $$ = op_var(OP_ADD, $1, $3); }  
| Terme tSUB Terme { $$ = op_var(OP_SUB, $1, $3); }  
| Terme tMUL Terme { $$ = op_var(OP_MUL, $1, $3); }  
| Terme tDIV Terme { $$ = op_var(OP_DIV, $1, $3); }  
| tPO Terme tPF { $$ = $2; }  
| tMUL tNAME { $$ = get_value_pointer($2); }  
| tNAME { $$ = get_var($1); }
```

Boucles et conditions

Pour pouvoir rentrer dans une condition ou dans une boucle, on doit vérifier si cette dernière est vraie ou non, et cela se fait par la règle “Verification” qui contiendra l’adresse de la variable qui contient le résultat. Cette dernière sera directement utilisée par les instructions de saut selon l’instruction suivante

La condition (if) et la boucle while

Si la condition n’est pas vérifiée, alors on va directement sauter à la fin du bloc d’instruction du if. Cette valeur n’étant pas connue d’avance, il fallait la stocker quelque part. Nous avons donc utilisé soit un token soit une règle (mid-rule), chaque nouvelle instruction incrémente un compteur et on la récupère à la fin du bloc pour la mettre à jour avec patch la condition JMF. On continue ensuite avec les autres instructions.

La logique est la même avec la boucle while mais avec le début de la boucle qui sera stocké pour le jmp à la fin de la boucle, et un jmf au début en cas de condition non vérifiée.

If-else

Si la condition n’est pas remplie, alors on JMP directement à la fin du bloc et si il y a un else, on fait un jump vers la fin de celui-ci dans le cas où on est rentré dans le if (la logique étant la même de stocker dans une règle cette valeur avant de la mettre à jour

Écriture dans le fichier assembleur

Pour pouvoir mettre à jour nos lignes d'assembleur, nous avons créé un tableau d'instructions que nous remplissons à chaque écriture d'instruction. Quand on arrive à la fin d'un bloc, on met à jour les numéros pour les sauts conditionnels à l'aide de patch et à la fin de notre main, on génère notre fichier assembleur .ea avec chaque ligne, une instruction.

Conception du microprocesseur

La conception du microprocesseur s'est faite avec le langage VHDL. Nous avons commencé par faire tous les composants séparément puis ensuite nous avons fait le chemin de données pour tout relier.

Tester le microprocesseur

Pour tester notre programme, il suffit d'entrer les instructions dans le fichier "InstructionMemory.vhd", puis de lancer la simulation. On regarde ensuite si les registres et la mémoire ont les bonnes valeurs pour les instructions demandées.

Unité Arithmétique et Logique (UAL)

Cette unité permet de faire une opération arithmétique sur deux entiers. Elle prend donc en entrées deux signaux pour les entiers, et un signal pour l'opération à effectuer. On va retrouver une sortie S pour le résultat, et plusieurs bits de sortie pour la retenue de l'addition (C), si le résultat vaut 0 (Z), si le résultat est négatif (N) et si le résultat ne peut pas s'écrire sur 8 bits (O).

Dans l'architecture, nous avons fait un "case" avec Ctrl_Alu pour séparer les différentes opérations.

```
case Ctrl_Alu is
  when "001" => ... – ADD
  when "011" => ... – SUB
  when "010" => ... – MUL
  when "100" => ... – DIV
  when others => S <= "00000000";
end case;
```

L'addition

Nous faisons l'addition sur 9 bits afin de savoir si le résultat dépasse 8 bits ou non. Pour cela, nous avons créé une variable "tmp_add" qui prend la valeur de la somme des entrées A et B converties en SIGNED. Ensuite, nous testons si le 9ème bit est à 1. Si c'est le cas, alors on dépasse la capacité et on met le bit de retenue à 1.

La soustraction et la division

La soustraction et la division ne posent pas de problème en particulier. On se contente de faire l'opération avec les entrées converties en SIGNED puis on met le résultat dans la sortie.

La multiplication

La multiplication doit se faire sur 16 bits. Nous créons pour cela une variable comme pour l'addition afin de rentrer le résultat temporaire. Il faut maintenant tester si le résultat est

supérieur à 128. Tester uniquement le 9ème bit ne suffit pas car ceux d'après peuvent être à 1 et lui à 0. Nous entrons ensuite les résultats dans les bits de sorties.

Banc de registres

La particularité de notre banc de registre est qu'il est capable de lire deux registres à la fois. Nous créons pour cela deux signaux d'entrées qui correspondent aux numéros des deux registres à lire. Nous créons également deux sorties QA et QB qui correspondent aux valeurs des deux registres lus. Pour l'écriture dans un registre, un signal d'entrée correspond au numéro de registre dans lequel il faut écrire, une correspond à la valeur à écrire et un bit vaut 1 s'il faut écrire et 0 sinon.

Les registres sont enregistrés dans un tableau d'éléments de 8 bits chacun. Il y en a au total 16. Leur adresse peut donc être écrite sur 4 bits.

```
type array_logic_vector is array (0 to 15) OF STD_LOGIC_VECTOR(7 downto 0);
signal registers_tab : array_logic_vector;
```

Avec cette structure l'accès au registre i peut simplement se faire avec "registers_tab(i)".

Mémoire pour les données

La méthode pour stocker les données est la même que pour les registres. Elles sont dans un tableau d'éléments de 8 bits. Cette fois-ci, les adresses étant sur 8 bits, il y a 256 éléments. La principale différence avec les registres est qu'on ne peut pas lire et écrire simultanément. Il y a, pour différencier ces deux opérations, un bit 'RW' en entrée qui vaut 1 pour lire et 0 pour écrire. Il faut donc dans le code tester à chaque utilisation de la mémoire si ce bit vaut 1 ou 0.

```
if (RW = '1') then – READ
    OUTPUT <= memory_tab(TO_INTEGER(UNSIGNED(ADDR)));
else – WRITE
    memory_tab(TO_INTEGER(UNSIGNED(ADDR))) <= INPUT;
end if;
```

On retrouve également un bit RST qui permet de remettre toute la mémoire avec la valeur 0 lorsque celui vaut 0.

```
if (RST = '0') then
    OUTPUT <= "00000000";
    memory_tab <= (others => "00000000");
end if;
```

Mémoire pour les instructions

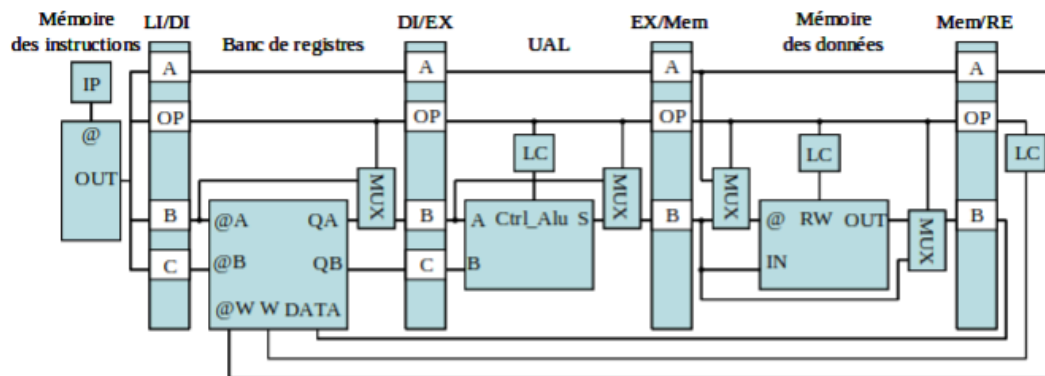
La mémoire pour les instructions va simplement chercher l'instruction à l'adresse donnée en entrée. Les instructions sont comme pour les registres et la mémoire dans un tableau d'éléments de 8 bits. Accéder à l'instruction à l'adresse ADDR se fait donc comme ceci :

```
OUTPUT <= instruction_tab(TO_INTEGER(UNSIGNED(ADDR)));
```


Chemin de données

Faire le lien entre les composants

Le chemin de données va faire un lien entre chaque composant qui compose notre microprocesseur. Il est divisé en quatre parties. Premièrement, la partie qui lit les instructions et les décode, puis celle qui exécute les instructions, celle qui écrit dans la mémoire et enfin la partie qui écrit dans les registres.



Dans VHDL, chaque partie correspond à un processus. Tous ces processus sont coordonnés par la même horloge. Entre chaque partie, des données peuvent passer sans entrer dans le composant. On a pour cela créé des signaux intermédiaires qui sont des STD_LOGIC_VECTOR de 8 bits. Ensuite, chaque partie peut traiter chaque instruction de notre jeu d'instruction grâce à des conditions et à un signal qui se transmet qui correspond à l'opération à effectuer.

Ajout de l'instruction NOP

Notre implémentation ne donne pas la possibilité d'enchaîner toutes les instructions. En effet, l'écriture dans les registres se fait à la fin du jeu de données. Il est donc possible qu'une autre instruction lise dans un registre avant qu'il soit écrit. Par exemple avec ces instructions :

```
AFC r1 0x1 // Ecrire 0x1 dans le registre r1
AFC r2 0x2 // Ecrire 0x2 dans r2
ADD r3 r1 r2 // Mettre le résultat de r1 + r2 dans r3
```

Avec ces instructions, r3 va lire le registre r2 lorsque celui-ci va se trouver dans la partie EX/Mem du microprocesseur. r2 ne va donc pas avoir la valeur 0x2. Nous devons donc pour cela faire "attendre" l'instruction d'addition. Nous avons donc ajouté une instruction NOP qui possède le code 0. Cette instruction va elle aussi passer dans les 4 parties du microprocesseur mais ne va rien faire. On va alors faire passer un cycle d'horloge pour que l'instruction précédente puisse terminer. Nos instructions ci-dessus devraient donc s'écrire :

```
AFC r1 0x1 // Ecrire 0x1 dans le registre r1
AFC r2 0x2 // Ecrire 0x2 dans r2
NOP
NOP
NOP
```

ADD r3 r1 r2 // Mettre le résultat de r1 + r2 dans r3

Nos tests

Nous avons réunis plusieurs tests dans ces instructions :

```
instruction_tab(1) <= "00000110"&"00000010"&"00000001"&"00000000"; -- Mettre 1 dans r2
instruction_tab(2) <= "00000110"&"00000011"&"00000010"&"00000000"; -- Mettre 2 dans r3
-- NOP
-- NOP
-- NOP
instruction_tab(6) <= "00000001"&"00000100"&"00000011"&"00000010"; -- r2 + r3 dans r4
-- NOP
-- NOP
-- NOP
instruction_tab(11) <= "00000010"&"00000101"&"00000100"&"00000011"; -- r4 * r3 dans r5
-- NOP
instruction_tab(13) <= "00000101"&"00000110"&"00000010"&"00000000"; -- Copier r2 dans r6
-- NOP
instruction_tab(15) <= "00001101"&"00000111"&"00000001"&"00000000"; -- Load 0x1 (8) dans r7
-- NOP
instruction_tab(17) <= "00001101"&"00001000"&"00000011"&"00000000"; -- Load 0x3 (11) dans r8
-- NOP
instruction_tab(19) <= "00001110"&"00000101"&"00000100"&"00000000"; -- STR r4 (3) dans 0x5
-- NOP
instruction_tab(21) <= "00001110"&"00000100"&"00000101"&"00000000"; -- STR r5 (6) dans 0x4
```

Voici la valeur initial de la mémoire :

memory_tab[0:256][7:0]	00,08,04,0a,00,
> [0][7:0]	00
> [1][7:0]	08
> [2][7:0]	04
> [3][7:0]	0a
> [4][7:0]	00
> [5][7:0]	00
> [6][7:0]	00
> [7][7:0]	00
> [8][7:0]	00
> [9][7:0]	00
> [10][7:0]	00
> [11][7:0]	00
> [12][7:0]	00
> [13][7:0]	00
> [14][7:0]	00
> [15][7:0]	00
> [16][7:0]	00

Voici les valeurs des registres (à gauche) et de la mémoire(à droite) à la fin de ces instructions :

🔍 registers_tab[0:15][7:0]	UU,UU,01,02,03,	🔍 memory_tab[0:256][7:0]	UU,08,04,0a,06,0
> 🔍 [0][7:0]	UU	> 🔍 [0][7:0]	UU
> 🔍 [1][7:0]	UU	> 🔍 [1][7:0]	08
> 🔍 [2][7:0]	01	> 🔍 [2][7:0]	04
> 🔍 [3][7:0]	02	> 🔍 [3][7:0]	0a
> 🔍 [4][7:0]	03	> 🔍 [4][7:0]	06
> 🔍 [5][7:0]	06	> 🔍 [5][7:0]	03
> 🔍 [6][7:0]	01	> 🔍 [6][7:0]	00
> 🔍 [7][7:0]	08	> 🔍 [7][7:0]	00
> 🔍 [8][7:0]	0a	> 🔍 [8][7:0]	00
> 🔍 [9][7:0]	UU	> 🔍 [9][7:0]	00
> 🔍 [10][7:0]	UU	> 🔍 [10][7:0]	00
> 🔍 [11][7:0]	UU	> 🔍 [11][7:0]	00
> 🔍 [12][7:0]	UU	> 🔍 [12][7:0]	00
> 🔍 [13][7:0]	UU	> 🔍 [13][7:0]	00
		> 🔍 [14][7:0]	00
		> 🔍 [15][7:0]	00
		> 🔍 [16][7:0]	00